

The Vault

Logins, bank details, cards, IDs and notes, with photos, locked behind one master password. Everything is encrypted on the phone and never leaves it: no account, no cloud, no network.

0 network requests (CSP <code>connect-src 'none'</code>)	600,000 PBKDF2-SHA256 rounds per unlock	2 builds: one HTML file, or installable PWA	30 days in the trash before anything is really gone
--	--	--	--

THE STACK

Browser-native, nothing rolled by hand

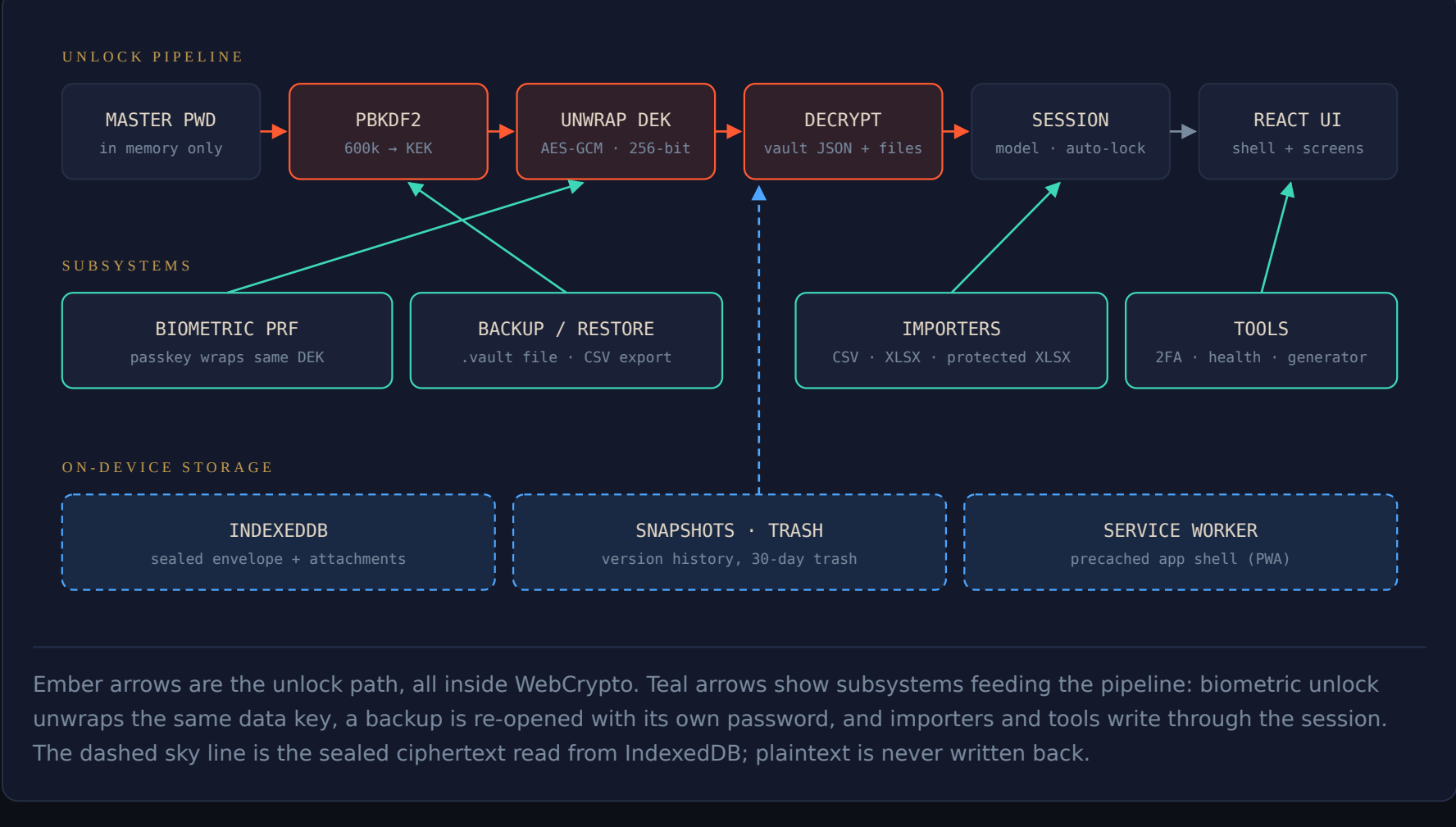
All cryptography is WebCrypto. The only runtime libraries are React and an icon set; everything else, including the Excel and OLE parsers, is local TypeScript.

UI React 19 Shell with tabs, search, bulk select; screens for items, import, health, settings.	LANGUAGE TypeScript Strict types across core and UI, checked with <code>npm run typecheck</code> .	CRYPTO WebCrypto PBKDF2 → KEK wraps a random 256-bit DEK; AES-GCM seals data, files and snapshots.	STORAGE IndexedDB Sealed envelope, attachments, snapshots; <code>localStorage</code> fallback, persistence requested.
UNLOCK Passkey PRF Fingerprint unlock wraps the same DEK with a WebAuthn PRF-derived key (installed app only).	BUILD Vite 8 <code>vite-plugin-singlefile</code> for <code>password-vault.html</code> ; a second mode emits the PWA.	OFFLINE Service worker Generated from <code>sw.template.js</code> , precaches the hashed assets.	TESTS Vitest · Playwright Unit tests on jsdom + fake IndexedDB; real-browser flows at phone size (412×915).

ARCHITECTURE

From master password to screen

The master password never touches storage. It derives a key-encryption key that unwraps the data key; only then is the vault decrypted into memory.



HOW IT WAS BUILT

Six phases

The order the pieces went in, from the crypto core outward. Open a phase to see its steps.

01	Crypto core Key wrapping the way mainstream managers do it	CRYPTO.TS ▾
1.1	KEK from password PBKDF2-SHA256 at 600,000 rounds with a random salt; 100,000 is the floor for old files.	
1.2	Random data key A 256-bit DEK is wrapped by the KEK, so a password change re-wraps one key instead of re-encrypting everything.	
1.3	Seal everything AES-GCM with fresh IVs over the vault JSON, each attachment and each snapshot.	
1.4	v2 envelope <code>format: "pwvault", v: 2</code> with KDF params, wrapped keys, revision counter and timestamp.	
1.5	v1 read path The original single-file format still opens read-only and is migrated forward.	
02	Storage & session Safe saves, locking and history	SESSION.TS ▾
2.1	IndexedDB store Envelope, attachments and snapshots, with a <code>localStorage</code> fallback and a persistent-storage request.	
2.2	Verified writes Every save is read back and checked; a failed check raises instead of silently losing data.	
2.3	Multi-tab safety A revision counter detects a vault changed in another tab and refuses to overwrite it.	
2.4	Auto-lock Locks after inactivity or when the app is hidden; copied secrets are cleared from the clipboard.	
2.5	Snapshots & trash Version-history snapshots, earlier passwords kept per item, 30-day trash and undo.	
03	App shell & screens Phone-first React UI	UI/ ▾
3.1	Shell Category tabs, search, favorites and multi-select for bulk move or delete.	
3.2	Item types Logins, bank, cards, IDs and notes, plus free-form extra fields such as PIN or sort code.	
3.3	Attachments Photos and PDFs shrunk on-device, then sealed like every other byte.	
3.4	Tools Built-in TOTP codes, a password health check (weak, reused, old) and a character or word generator.	
3.5	Theme Dark and light themes sized and tested at 412×915.	
04	Importers Bring the old spreadsheet in, pictures and all	IMPORTERS/ ▾
4.1	CSV & manager exports Chrome, Bitwarden, 1Password and Firefox layouts mapped onto vault fields.	
4.2	XLSX parser Zip and sheet XML read locally; every tab becomes a category and pictures stay with their row.	
4.3	Password-protected Excel Compound-file (CFB) reader plus Office Agile and Standard decryption in <code>officecrypto.ts</code> .	
4.4	Hostile-file limits 64 MB per part, 256 MB total, 4 M cells and 300 pictures, so zip bombs and lying sizes fail cleanly.	
05	Two builds, one source Single file and installable PWA	VITE ▾
5.1	Single-file mode <code>password-vault.html</code> ; everything inlined, open it straight from disk.	
5.2	PWA mode Hashed assets, manifest, icons and a generated service worker for offline launch.	
5.3	Fingerprint unlock WebAuthn PRF passkey, available once installed on an https origin.	
5.4	Locked-down headers <code>_headers</code> sets a strict CSP, no framing, no referrer, and HSTS.	
5.5	Release script <code>npm run release</code> builds both and copies them to the repo root for static hosting.	
06	Hardening & verification Regression suite and screenshot tour	TESTS/ ▾
6.1	Unit coverage Crypto, session, model, importers, Office decryption, strength, TOTP and health.	
6.2	Browser flows Playwright at phone size: setup, edit, import, backup and restore, PWA install.	
6.3	v1 → v2 migration The original app is a fixture; its vault must open and move across intact.	
6.4	Hostile fixtures Generated bombs, huge rows and web-linked images guard the import limits.	
6.5	Screenshot tour <code>SHOTS=1</code> run produces the README images from the real app.	

TEST STRATEGY

What keeps it honest

Run from `app/`: `npm test`, `npm run typecheck`, `npx playwright test`.

UNIT <code>vitest · jsdom · fake-indexeddb</code> → crypto: wrap, unwrap, wrong password, iteration floor → session: saves, conflicts, auto-lock, snapshots → officecrypto: Agile and Standard encrypted workbooks → importers, model, strength, TOTP, health	HOSTILE INPUTS <code>python fixture generators</code> → Zip bombs, including ones that lie about their size → Huge rows and pictures anchored under headings → Web-linked images that must never be fetched → Workbooks encrypted by Excel and by LibreOffice
INTEGRATION <code>playwright · chromium · 412×915</code> → Flows: setup, add, edit, search, bulk actions, trash → Import: real spreadsheets end to end → Migration: v1 single-file vault to v2 → PWA install, offline launch, regressions	MANUAL on a real Android phone → Install from Chrome and turn on fingerprint unlock → Back up, then restore into the installed app → Airplane mode: open, unlock, edit, lock → Switch apps and confirm it locks itself

IN THE REPO

Where things live

<code>password-vault.html</code>	The whole app in one file. Download and open.
<code>index.html · assets/ · sw.js</code>	The installable build; host this folder.
<code>_headers</code>	Strict CSP and security headers for Netlify.
<code>app/src/core/</code>	Crypto, session, storage, importers, TOTP, health.
<code>app/src/ui/</code>	React shell and screens.
<code>app/tests/</code>	Unit, end-to-end and fixture generators.